

(15 pts Questão 1) Configure o registro T2CON de modo que o timer 2 do PIC esteja desligado, operado com postscale de 1 para 2 e prescaler 16. Escreva o valor para T2CON em binário, decimal e hexadecimal.

REGISTER 13-1: T2CON: TIMER2 CONTROL REGISTER

U-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
—	T2OUTPS3	T2OUTPS2	T2OUTPS1	T2OUTPS0	TMR2ON	T2CKPS1	T2CKPS0
bit 7							bit 0

bit 7 **Unimplemented:** Read as '0'

bit 6-3 **T2OUTPS3:T2OUTPS0:** Timer2 Output Postscale Select bits
 0000 = 1:1 Postscale
 0001 = 1:2 Postscale
 •
 •
 •
 1111 = 1:16 Postscale

bit 2 **TMR2ON:** Timer2 On bit
 1 = Timer2 is on
 0 = Timer2 is off

bit 1-0 **T2CKPS1:T2CKPS0:** Timer2 Clock Prescale Select bits
 00 = Prescaler is 1
 01 = Prescaler is 4
 1x = Prescaler is 16

T0CON = 0b000101x -> 0b0000 1010 -> 0x0A -> 10

(20 pts Questão 2) Crie um **programa** que leia um valor de distância de um sensor ultrassônico na entrada analógica e apresente a distância no barramento de LED's (porta D). Este sensor consegue identificar distâncias de 40 cm à 200 cm. Quando há um objeto à 40 cm a saída do sensor apresenta 0 volts (0 no AD). Quando o objeto está a distância de 200 cm a saída fica saturada em 5volts (1024 no AD). O sensor possui uma resposta linear. Pode-se utilizar a biblioteca "adc.h". O programa deve deixar todos os leds acesos quando o sensor indicar objeto à 40 cm e apagar um led a cada 20cm, de modo que todos os leds fiquem apagados quando o objeto estiver à 200cm ou mais.

```
//adc.h
void InicializaADC(void);
unsigned int LeValorAD(void);
```

```
#include "adc.h"
#include "basico.h"
#include "config.h"
```

```
void main (void){
    char i;
    int dist;
    InicializaADC();
    TRISB = 0x00;
    for(;;){
        dist = LeValorAD();
        //transforma a escala de 0 à 1024 para 0 à 8
        dist = dist * 8 / 1024;
        PORTB = 0x00; //acende os 8 leds
        for(i=0;i< dist; i++){
            //Apaga uma quantidade de leds relativa a distância
            BitClr (PORTB, i);
        }
    }
}
```

```
}  
}
```

(20 pts Questão 3) Escreva uma **função** que faça o tratamento da interrupção de recepção de dados da serial. Esta função será responsável por ler o valor recebido da serial no registro **RCREG** e armazená-lo em uma variável global. A flag responsável por indicar que chegou uma informação pela serial é o **bit 3** do registro **PIR1**.

```
//variavel global  
static unsigned char dummy;  
void nome_qualquer (void) __interrupt 1{  
    if (BitTst(PIR1, 3)){  
        dummy = RCREG;  
    } else {  
        dummy = 0;  
    }  
}
```

(20 pts Questão 4) Monte um **programa** que: 1) recebe um caracter via comunicação serial e se este valor for uma letra, exibe no LCD. Caso este caracter for um número, ele deve ser exibido no display de 7 segmentos.

```
unsigned char RecebeSerial(void);  
void AtualizaDisplay(void);  
void MudaDigito(unsigned char valor, unsigned char posicao); //valor em número normal  
void EnviaDadosLCD(unsigned char valor); //valor em codificação ASCII
```

```
#include "lcd.h"  
#include "disp7seg.h"  
#include "serial.h"  
#include "basico.h"  
#include "config.h"  
  
void main (void){  
    char txt;  
    InicializaADC();  
    InicializaLCD();  
    InicializaDisplays();  
    for(;;){  
        txt = RecebeSerial();  
        if(txt>= '0' && txt <= '9'){  
            MudaDisplay(txt, 0);  
        }  
        if(txt>= 'A' && txt <= 'Z'){  
            EnviaDadosLCD(txt, 0);  
        }  
        AtualizaDisplay();  
    }  
}
```

(25 pts Questão) Na porta B estão ligadas 4 chaves nos bits 0,1,2 e 3, e quatro leds nos bits 4,5,6 e 7. Faça uma **biblioteca** chamada "interface" com quatro funções: 1) Inicializa os registros pertinentes à porta B, 2) Realiza o debounce das teclas, 3) Retorna o número da chave que está ligada, 4) Recebe um char indicando qual led deve ser aceso, 5) Recebe um char indicando qual led

deve ser apagado.